

CO3219 Internet and Cloud Computing

Coursework: Cloud System Design and Evaluation

Name: Kyle D S Dhesi

ID: 209026738

Group 6

Cloud System Design and Evaluation

A private cloud is an environment in which hardware and software resources are exclusive and only accessible to a single user. By designing a private cloud according to the five principles of cloud architecture, an individual can develop a private cloud suitable for an organisation.

Private Cloud Architecture

Private clouds make use of technologies that allow the user to provide and construct virtual servers. To optimize utilisation of resources, deployment of on demand computing resources allow for fast scale response to increased usage and traffic. Private clouds are based off the three technologies:

1. **Virtualisation** – Allows IT resources to be taken from physical hardware and pooled into a resource pool of computing, storage, memory and networking which can be divided among multiple virtual machines.
2. **Management software** – Administrators have control over applications and infrastructure running on the software.
3. **Automation** – Speeds up server provision, which removes the need of human intervention of completing this manually and constantly.

Benefits of Private Cloud

Some benefits of private clouds consist of:

- Improved insight of security and user access control
- Flexibility of customisation of software and hardware resources in response to demand
- Control over the inclusion of required hardware and software

Objective

The objective of our project is to design, develop and deploy a whiteboard application onto a private cloud. The whiteboard application must be able to run on two different nodes within the private cloud, and consist of benefits such as, reliable, secure, elastic and high performing.

Design and Evaluation of Private Cloud

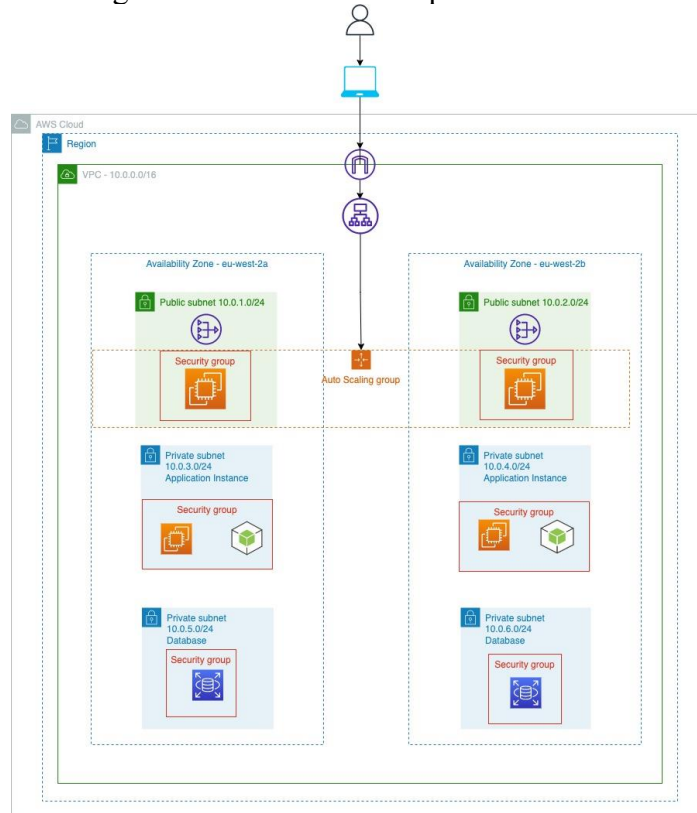
We first compared the differences between Amazon Web Services (AWS), Microsoft Azure and Google Cloud Platform (GCP), and concluded using AWS. AWS consists of an improved infrastructure, scalability and greater resources opposed to Azure; offers a wider range of services in comparison to GCP. Furthermore, AWS is easy to use, high-performing and reliable, all of which are beneficial to our cloud.

The following are our product/service choices for our private cloud:

- Amazon Virtual Private Cloud
- Amazon EC2
- Amazon ELB

- AWS Identity and Access Management
- Amazon RDS

Below is a diagram outlining the architecture of our private cloud:



We intended on designing our private cloud to conform to AWS Well-Architected framework.

Consistency:

To achieve consistency within our private cloud, we have implemented databases, to ensure necessary data is stored and recalled whenever is needed.

Scalability:

We made use of auto scaling groups to cater for increase or decrease in workload, to keep a consistent performance within our application. Furthermore, the load balancer distributes traffic to accordingly to EC2 instances in either availability zone.

Low latency:

To ensure low latency, we have chosen region 'eu-west-2', as this allows the application to work best and provides the most optimal network latency within our application. This region is the closet to our location therefore network packets can travel faster.

Technologies used within the private cloud architecture:

Amazon Virtual Private Server: Enables the launch of instances within a defined private network.

AWS Internet Gateway: A horizontal scale component that enables communication between our VPC and internet.

AWS Subnets: A set of IP addresses within the VPC where resources such as EC2 instances can be launched.

AWS NAT Gateway: NAT gateway is used to allow instances within a private subnet to connect to services outside our VPC.

AWS Security Group: Initial configuration to control desired traffic in and out of instances.

Amazon EC2: A cloud computing platform that can be auto-scaled based off demand – virtual machines are able to run applications

Node.js: An open-source server environment which can generate dynamic page content.

Amazon RDS: A collection of services that setup, operate and scale databases in the cloud, and improve overall performance.

Implementation of Private Cloud

Upon designing our architecture, it was time for us to implement our private cloud. The first step was initiating our VPC.

VPC Implementation

The name of the VPC is, 'whiteboard-application', and we assigned a IPv4 CIDR block of 10.0.0.0/16. This allows for 65,536 hosts and has a subnet mask of 255.255.0.0. We then moved onto creating our subnets. In total, six subnets were created – two public and four private. The IPv4 CIDR block ranges from 10.0.1.0/24 to 10.0.6.0/24 between all subnets, and the subnets are equally divided between our two availability zones.

☐	Name	Subnet ID	State	VPC	IPv4 CIDR
☐	Private-1C-DB	subnet-0c609c1eda50308a9	Available	vpc-01a120f8e3ec35922 whi...	10.0.5.0/24
☐	Private-2C-DB	subnet-0086901f65500afb0	Available	vpc-01a120f8e3ec35922 whi...	10.0.6.0/24
☐	Private-2B-Applicat...	subnet-057b13c0b061920bd	Available	vpc-01a120f8e3ec35922 whi...	10.0.4.0/24
☐	Public-2A	subnet-03fad30e76bdaea38	Available	vpc-01a120f8e3ec35922 whi...	10.0.2.0/24
☐	Private-1A-Applicat...	subnet-0fa15bba9d04d2cdf	Available	vpc-01a120f8e3ec35922 whi...	10.0.3.0/24
☐	Public-1A	subnet-0697280ed511bf9ef	Available	vpc-01a120f8e3ec35922 whi...	10.0.1.0/24

- Public-1A and Public-2A are our public web facing subnets, which host EC2 instances
- Private-1A-Application and Private-2B-Application are private backend facing subnets, that will too host EC2 instances
- Private-1C-DB and Private-2C-DB are also private subnets and host our database

An internet gateway was created and attached to our VPC to connect to the internet. We have created two route tables – Public-RT and Private-RT, to direct traffic; a NAT gateway was made and linked to our Private-RT to allow for communication for services within private subnets.

The final step in configuring our VPC was creating our ‘Security Groups’. In total, ten security groups were made, to help control the flow of traffic in and out of elements within our cloud.

☐	Name	Security group ID	Security group name	VPC ID
☐	–	sg-0dd4cc40450aa96e0	Private App	vpc-01a120f8e3ec35922
☐	–	sg-05548901a4230e326	ec2-rds-1	vpc-01a120f8e3ec35922
☐	–	sg-04441ac5d0365d4d3	ec2-rds-2	vpc-01a120f8e3ec35922
☐	–	sg-0108572c1df336e3a	database	vpc-01a120f8e3ec35922
☐	–	sg-0d9c80cc70865fb3b	rds-ec2-2	vpc-01a120f8e3ec35922
☐	–	sg-0c5e395cfd6691c85	rds-ec2-1	vpc-01a120f8e3ec35922
☐	–	sg-05b6809cb9930fc16	default	vpc-01a120f8e3ec35922
☐	–	sg-06f20cfd80a7db94	Public Web Access	vpc-01a120f8e3ec35922
☐	–	sg-0b3c0fee44fe5dea5	database2	vpc-01a120f8e3ec35922
☐	–	sg-045dcbad342799210	lb-sg	vpc-01a120f8e3ec35922

- Private App – Allows for SSH, HTTP and SQL connectivity
- Public Web Access – Allows access to web
- LB-SG allows traffic to port 80
- RDS-EC2 allows for connection between EC2 instance and database

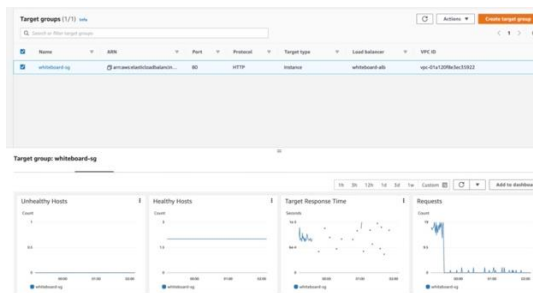
EC2 Instances

After setting up our VPC, we created our four instances:

- Public1A-Web
- Private1A-Application
- Public2B-Web
- Private2B-Application

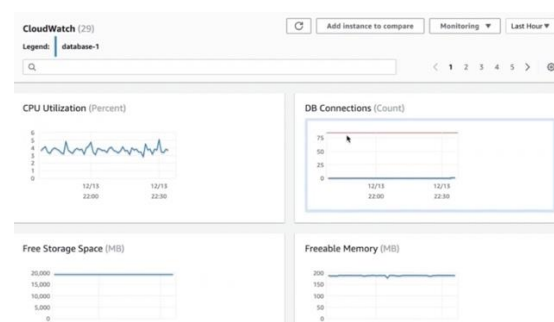
These instances are divided into separate availability zones; web instances are presentation tier for the whiteboard, and application tier are backend.

To improve network distribution and scalability, an application load balancer was connected to both availability zones, and instances to distribute traffic. In order to monitor this, we make use of the ‘monitoring’ tab within our target groups. We have two healthy hosts, meaning our load balancer is operating successfully.



Furthermore, to improve fault tolerance and scalability, auto scaling group is used which contains our EC2 instances.

Like our instances, databases have also been created in both availability zones to help achieve consistency. To connect our database to our subnets/instances, we used the terminal and EC2 Instance Connect. Finally, to monitor our database, CloudWatch dashboard is used.



To launch the whiteboard application onto the web, use of the user's terminal or EC2 Instance Connect is required. The user must first install Node.js, which offers high speed and performance. They can clone the whiteboard application onto their virtual directory and run it on port 3001. Other users can use the same application, and changes such as drawing a line, are updated on both applications.

```

Last login: Tue Dec 13 22:44:14 2022 from ec2-3-8-37-28.eu-west-2.compute.amazonaws.com

 _ _ _ _ _
| | | | |
|_|_|_|_|_|

Amazon Linux 2 AMI

https://aws.amazon.com/amazon-linux-2/
[ec2-user@ip-10-0-2-226 ~]$ cd whiteboardapp_aws
[ec2-user@ip-10-0-2-226 whiteboardapp_aws]$ npm install

up to date, audited 65 packages in 1s

11 vulnerabilities (1 low, 2 moderate, 8 high)

To address all issues, run:
  npm audit fix --force

Run `npm audit` for details.
[ec2-user@ip-10-0-2-226 whiteboardapp_aws]$ node index.js
listening on port 3001

```

Implementation and Deployment of Whiteboard Application

To create the whiteboard application, it was developed on VisualStudioCode as it offered an easy-to-use platform to develop HTML and CSS.

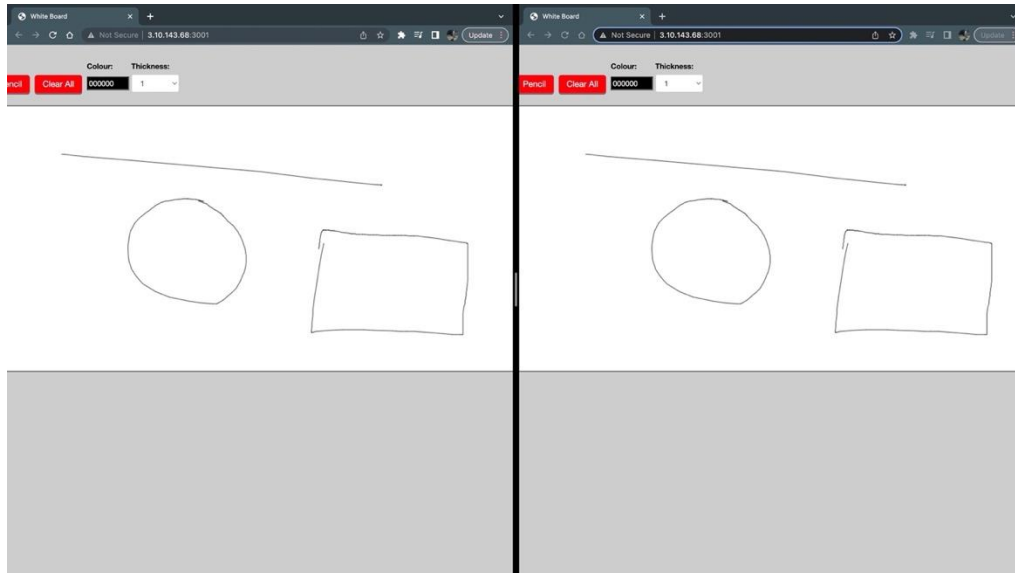
We made use of the following languages:

- HTML
- CSS

- Javascript

As previously mentioned, to configure and deploy this application onto the private cloud, the user must install Node.js using their terminal and clone the whiteboard application into their directory and run the application on port 3001.

Interface screenshots of multiple users being able to draw on the same canvas.



Key features implemented:

- User can change the thickness of the pencil allowing for thicker lines/shapes to be drawn
- User can change the colour of the pencil
- The 'clear all' button deletes the canvas for both users synchronously
- The application is highly available on multiple systems through the use of nodes

Consistency:

The application is consistent as it's synchronous, any changes made to one canvas, will be updated to the other.

Low latency:

Any changes made to the application on a user's computer (i.e. drawing a shape) is updated on the other user's computer as the drawing is taking place – there is no output delay between the two applications.

Scalability:

The whiteboard application can be hosted within our two different availability zones, and traffic is distributed accordingly via our load balancer and scaling group. The functionality of the whiteboard is the same in each availability zone.

Development of whiteboard:

When developing our whiteboard, we started off by designing the application. Our intentions were to keep the application basic with minimal functionality, so it can operate best within our private cloud.

We started off coding by creating our HTML page and the respective CSS sheet. After, we moved onto adding our features – clear all button, change pen thickness and pen colour. Functions were included to allow for the user to draw onto the canvas using their mouse.

Then we added sockets as they are beneficial for efficient information transfer between the network.

Finally, an index.js file was required as this is responsible for the start-up and running of our application using Node.js.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>White Board</title>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <meta name="description" content="A Collaborative White board">
  <meta name="author" content="devan">
  <style type="text/css">
    #container { position: relative; }
    #container {
      width: 1000px;
      height: 500px;
      background-color: #fff;
      border-radius: 5px;
    }
    body {
      background: #rgb(205, 205, 205) !important;
    }
    #imageView { border: 1px solid #000; }
    #imageTemp { position: absolute; top: 1px; left: 1px; }
    canvas {
      border-radius: 5px;
      cursor: url(.../img/pen.png), crosshair;
    }
    #text_tool {
      position: absolute;
      border: 1px dashed black;
      outline: 0;
      z-index: 1000 !important;
      display: none;
      overflow: hidden;
      white-space: nowrap;
    }
    .container .btn.btn-sm {
      background: #rgb(255, 0, 0);
    }
  </style>
</head>
<body>
  <div class="container">
    <div class="imageView">
      <img alt="Whiteboard canvas" class="imageTemp" />
      <canvas class="canvas">
    </div>
    <div class="text_tool">
    </div>
    <div class="btn">
    </div>
  </div>
</body>
</html>
```

```
JS index.js
JS index.js > ...
1
2 const express = require('express');
3 const app = express();
4 const http = require('http').Server(app);
5 const io = require('socket.io')(http);
6 const port = process.env.PORT || 3001;
7
8 app.use(express.static(__dirname + '/public'));
9
10 function onConnection(socket){
11   socket.on('drawing', function(data){
12     socket.broadcast.emit('drawing', data);
13     console.log(data);
14   });
15
16   socket.on('Clearboard', function(data){
17     socket.broadcast.emit('Clearboard', data);
18     console.log(data);
19   });
20 }
21
22 io.on('connection', onConnection);
23
24 http.listen(port, () => console.log('listening on port ' + port));
25
26
```

Demonstration of Private Cloud

To develop the private cloud, an AWS Root account was created, and each member was assigned an IAM account.

Overview of my work within the project:

- Initialise and setup VPC with respective subnets, NAT gateways, and route tables

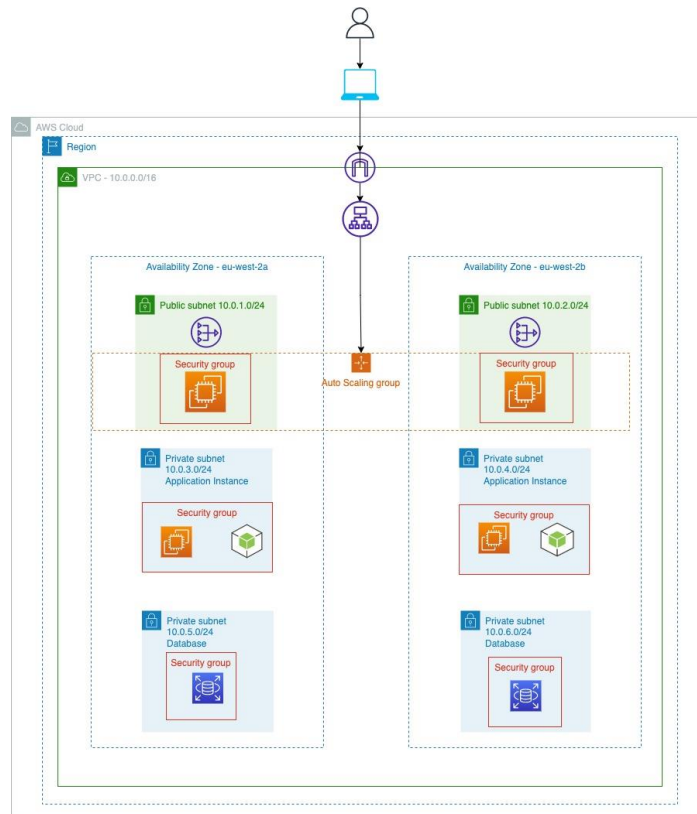
- Creation of EC2 instances with respective security groups, auto scaling group and application load balancer
- Creation of the database
- Design of architecture diagram
- Test and monitor application to ensure everything is running optimally and make any required changes

Contributions within the project:

- Worked alongside developer when coding the whiteboard application

Critical Review of Private Cloud

Reflections on design and implementation:



Our cloud was hosted on AWS opposed to Azure or Google Cloud Platform. This decision was based off AWS offering an easy-to-use platform with a range of helpful material online to help us develop a cloud that is high performing, available and flexible.

The use of an application load balancer was to improve efficiency and scalability of our private cloud, which distributes traffic to each availability zone.

Autoscaling groups allowed for optimization of applications via monitoring of resources and their changes in demand.

Security groups improved the security of our application as they controlled the flow of traffic in and out of our instances, databases and subnets.

The choice of EC2 instances allowed for easy and fast deployment of applications. Furthermore, it helps conform to the AWS architecture framework of being, reliable, secure and elastic. An alternative was Lightsail however it doesn't offer the same benefits as EC2.

Including a database improves the consistency of the application as data can be recalled whenever it is needed with ease.

Strengths of our private cloud:

- Scalable
- Secure
- High performing
- Low latency
- Highly available

Weaknesses of our private cloud:

- Consistency
- Simplicity
- Quality of service

Upon developing our private cloud, we solved many different challenges to help ensure our outcome is high performing, efficient and apply to the six pillars of AWS. A main challenge that we faced was including the database and connecting it to our instance; however, the addition of a database can improve the consistency. The inclusion of auto scaling groups, and balancer and EC2 instances helps to make our cloud scalable. Availability zones help keep latency low; therefore, data can transfer fast between the application. Furthermore, this makes our cloud available across more regions too. Security groups improve the security, and the use of EC2 instances helps to achieve all the above.

Improvements to our private cloud

To overcome our weaknesses, AWS provides a wide range of services which can be used to better our private cloud. To improve security, the use of 'Cognito', enables easy user access into the application. Another scaling group can be used within our private subnets, to further improve the running of the instances in this subnet and scalability. Multiple load balancers can be used to prevent failure caused from over trafficking. The service 'Elasticache' can be linked to our database, and this can upgrade our consistency and performance.

In conclusion, we have developed and deployed a private cloud that is able to host a whiteboard application using nodes, in different availability zones, where users can simultaneously interact with the application. Our cloud is scalable, highly available and has low latency.